

A CGP-based Logic Flow: Optimizing Accuracy and Size of Approximate Circuits

Augusto Berndt¹, Brunno A. de Abreu², Isac S. Campos¹, Bryan Lima¹,
Mateus Grellert¹, Jonata T. Carvalho¹, and Cristina Meinhardt¹

¹Departamento de Informática e Estatística, PPGCC, Universidade Federal de Santa Catarina - UFSC, Brazil

²Instituto de Informática, PGMicro, Universidade Federal do Rio Grande do Sul - UFRGS, Brazil
e-mail: augusto.berndt@posgrad.ufsc.br

Abstract— Logic synthesis tools face difficult challenges regarding algorithms for synthesizing circuits with increased inputs and complexity. Machine learning techniques shows to be an attractive option to improve electronic design tools. We explore Cartesian Genetic Programming (CGP) for logic optimization of exact or approximate Boolean functions in our work. The proposed CGP-based flow receives the expected circuit behavior as a truth-table and either performs the synthesis starting from random circuits or optimizes a circuit description provided in the format of an AND-Inverter Graph. The optimization flow improves solutions found by other techniques, using them for bootstrapping the evolutionary process. We use two metrics to evaluate our CGP-based flow: (i) the size defined by the number of AIG nodes or (ii) the circuit accuracy. We perform experiments on approximate circuits, and results show that the CGP-based flow provided at least 22.6% superior results when considering the trade-off between accuracy and size compared with two other methods that brought the best accuracy and size outcomes, respectively.

Index Terms— Logic Minimization; Evolutionary Computing; Machine Learning; Cartesian Genetic Programming; Multi-variable Optimization.

I. INTRODUCTION

The design of integrated circuits is performed by iteratively compiling, optimizing, and verifying the hardware description language of a circuit to achieve its physical representation layout. Logic synthesis is one of many processes composing the design flow of digital circuits. During logic synthesis, designers face the challenge of providing a suitable circuit mapped with relation to a library of cells.

The task of logic synthesis optimization aims at reducing the number of cells or logic depth of a Boolean function, mainly because these reductions are correlated with improvements in the circuits final area, power, and delay. The high increase in complexity of functions is the main challenge imposed on logic synthesis. It corresponds to the high number of inputs, which raises the number of combinations to be considered in logic minimization. Nowadays, it is common to deal with circuits with 64 or more inputs on arithmetic modules, leading to 2^{64} different input combinations.

Some well-known logic minimization techniques to simplify Boolean functions are the Algebraic method, Karnaugh map [1], Quine-McCluskey[2], and Espresso [3, 4]. However, these traditional methods have some limitations when dealing with circuits containing a high number of inputs[5]. These limitations can be restrictive for the logic synthesis of larger circuits.

The significant advances of Machine Learning (ML) motivates its usage on the logic synthesis of combinational circuits [6]. There are multiple ways to explore the use of ML techniques in the extensive design flow of integrated circuits, and there is still a lot to be discovered [7]. ML techniques have been applied to logic synthesis for multiple purposes. Some works use ML for recognizing the functionalities of logic circuits, which may assist in providing extra information on hardware safety and reverse engineering [8]. Other works seek to use ML to improve classic logic optimizations, as in [9], where the models are used to detect the most prominent locations for optimization. ML techniques can also be used for approximate logic synthesis and optimization [10] when considering error-resilient scenarios.

Approximate Computing is an alternative to make the design flow faster and obtain power-efficient solutions targeting error-tolerant applications. These applications have less strict accuracy requirements of the implemented functions. In these designs, some slight distortions in the output can be acceptable, making it possible to optimize the designs in other constraints like area, power, and timing [11].

Some solutions address the generation of a reasonably small logic circuit that attempts to generalize an entire input space based on selectively provided samples, targeting the best accuracy. This objective is proposed on the International Workshop on Logic and Synthesis (IWLS) Contest in 2020 [6]. The contests primary goal was to obtain the best accuracy with a limit of 5000 nodes. The number of inputs of the logic functions in the Benchmarks vary from 10 to 768. The solutions proposed for this contest have addressed the problem with ML techniques, mainly exploring Decision Trees (DTs) and Random Forests (RFs). We have proposed a population-based optimization method with Cartesian Genetic Programming (CGP).

Despite the fact that the main optimization target in the contest was accuracy, the challenges imposed are more complex and relevant to current open issues in the logic synthesis domain. These issues include the problem of dealing with a huge increase in logic complexity, or on the optimization of a logic function without having access to the complete truth table, keeping the traditional targets of area, power, and delay. An additional challenge revolves around the capability of traditional flows in dealing with approximate circuits synthesis, especially considering it may be impossible to find exact solutions due to resource limitations.

The aforementioned challenges and issues lead us to propose a CGP-based flow that seeks to optimize both accuracy and size. The results of this flow are two-fold: (i) it allows

us to further improve the solutions found by the other techniques used for bootstrapping the evolutionary process, and (ii) seek optimal circuits starting the CGP search from random (unbiased) individuals [12].

In this work, we extend the proposed flow, providing a detailed description of the algorithms for the Bootstrapped flow that explores the CGP technique. The CGP-based flow is applied as an optimization step in order to improve the trade-off between accuracy and size. The CGP-based flow optimizes the size concerning the number of AIG nodes, maintaining or improving the initial accuracy of the circuit. With this combination of optimization parameters, the CGP solution can provide circuits with a good and controllable trade-off between area and accuracy. Such explorations are current demands for error-tolerant applications that compromise the accuracy targeting power efficiency. The results show that our approach can present improvements in the AND-Inverter Graphs (AIGs) provided by the bootstrapping methods, reducing the number of nodes with some improvements on the accuracy.

This paper is organized as follows. Section II starts presenting the problem contextualization and previous work related to it. Section III presents a general description of the CGP technique. Section IV details our CGP-based flow and how it is able to optimize logic circuits. Section V presents the evaluations and results obtained. Section VI concludes the work, summarizing our main findings and contributions.

II. PROBLEM CONTEXTUALIZATION AND RELATED WORK

Nowadays, humankind is vigorously exploring creating mathematical models that generalize for specific problem solving through academic research and industry research and development centers. The high computing capacity of current devices enables the exploration of ML models on new challenges, such as the optimization of logic circuits. In this section, we present a brief background on logic optimization, along with previous works concerning the synthesis of logic circuits by trading exactness for the capability of generalization. In other words, we present works that apply logic optimization as a ML problem.

A logic optimization problem can be defined as: given a Boolean function $f : \mathbb{B}^n \rightarrow \mathbb{B}$, where $\mathbb{B}$ is given by the set $\{0, 1\}$, the goal is to synthesize a logic circuit that represents f minimizing its number of logic elements. We consider the problem at hand as a supervised ML challenge. A logic function f may be incompletely specified, and the objective is to generalize outside the specified set. Only a partial subset of the tuples $(\mathbb{B}^m, \mathbb{B})$, where $m < n$, including inputs and outputs, are known, also called as *care-set*. This subset may be represented by a binary truth table and the supervised ML problem dataset. By splitting the care-set into training and validation sets, it is possible to construct the circuit and assess its generalization capability. Thus, a ML algorithm may be applied to the training set, which partially describes the Boolean function behavior. We may assume that if the model can generalize for the validation set, it should generalize to the unspecified Boolean combinations to a certain extent. It is noteworthy that in this situation, we are relaxing

the issue of possible exponential combinations in a logic circuit, which is given by 2^n since we are working with only a subset of $\mathbb{B}^n$. Overall, we are trading exactness for the generalization of logic circuits, and we are intrinsically working with approximate circuits.

Classical logic synthesis techniques focus on optimizing logic circuits presenting exact solutions, in contrast to finding circuits capable of generalizing to unspecified Boolean behavior. Such techniques usually apply logic optimizations on logic functions in different formats, such as Boolean networks like AIGs [13], XOR-AND-Inverter Graphs (XAIG) [14], Majority-AND-Inverter Graphs (MIG) [15], lookup-tables (LUTs) [16, 17], among other data structures representing Boolean networks. A network of LUTs are adopted in the generalization attempt to iteratively learn the logic to be implemented [17, 18]. In [17] the learning is done for each layer in the network of LUTs, from input to output. The work mentions an open question as to how neural networks can generalize on data; it is believed that they might only memorize and still be able to generalize on unseen data. This open question motivates the implementation of a network of LUTs for generalization. Also, logic representations not based on Boolean networks can be considered, such as Binary Decision Diagram (BDD) [19] or SOPs, although they almost always are not able to generalize. Works attempting to find logic functions with generalization capabilities introduce the usage of distinct structures such as DTs or RFs [20, 18, 21], MLPs or neural networks [18].

At this moment, there is no universal approach capable of always finding the best solution to the problem of developing logic functions capable of generalizing. A combination of multiple different techniques is required. This is the case for the winning team of the IWLS 2020 contest [6], whose proposal explores a combined adoption of Espresso [3], multi-layer perceptrons (MLP), random forests, and LUT networks [18] to reach the maximum accuracy. Results found that MLPs and RFs generally presented better accuracy when compared to Espresso and LUT networks. Also, the MLP shows the potential to optimize linearly non-separable elements (like XORs and XNORs), while random forests were reasonable alternatives to complex functions [6].

In [22], the authors propose an improvement to the classical technique of resubstitution [23] for Boolean networks. Their approach proposes a way to find possible candidates to substitute a node without the limitation of cut size, which is usually limited to up to ten primary inputs in the classical approach. Their method consists of generating candidate sub-circuits for substitution through fixing a Boolean value and generating other Boolean combination possibilities or generating substitution candidates through the observability of the node. Based on the results obtained, they compare their approach with the classical resubstitution technique. They achieve considerably smaller circuits and use K-cuts ten times greater, although their approach increases runtime slightly. The concept proposed in this work is contemporary. It attempts to generalize new candidate sub-circuits for substitution but at the same time holds a classical aspect, maintaining the circuit's integrity by verifying the possible solutions, validating candidates with SAT.

The CGP technique was introduced by [24], where the evolutionary technique was able to discover simple arithmetic circuits. Since then, other works have also explored the implementation of Genetic Programming as a logic optimization technique, similar to the proposed approach presented herein. Usually, the main issue is to improve runtime. Methods seek ways to achieve convergence to a solution with oriented mutations, unlike completely stochastic mutations. For example, in the recent work [25], a semantical orientation is proposed for mutations during the CGP search. Their results show significant improvement in convergence time. However, their experimental benchmarks are limited to arithmetic logic operations. In [26] an improvement in the CGP's scalability is proposed by selecting sub-circuits to be mutated during generations. They provide two approaches to define sub-circuits to be resynthesized and evaluate their search performance. Their results focus on area reduction only and come with the cost of increased runtime requirement. The technique of selecting sub-circuits for optimization under CGP was further improved by selecting sub-circuits with the presence of reconvergent paths [27]. Their experiments were performed on mapped logic circuits and shows that attending reconvergent paths is beneficial for logic circuits evolutionary process.

An overview of the CGP adoption shows different approaches to obtain improvements [28]. Among these, some works explore the absence of the issue known as bloat in CGP [29], analyzing whether the increase in the number of inactive nodes results in no significant improvement in fitness. The work [28] even attends the exploration of specialized crossover techniques during CGP search. Another recent survey [30] compares different types of CGP implementations. It presents multiple CGP implementations not necessarily related to logic circuits, addressing their limitations and advantages with the CGP technique.

Recently, fast solutions were proposed to generate approximate solutions focusing on BDD minimization [31] or based on Decision Trees [20]. In [32], the logic regression problem is explored in a flow based on two-level minimization methods. This research highlights a fundamental drawback of the proposed method: the complexity of implementing circuits with a large number of linearly non-separable elements as found in XOR and XNOR functions. ALFANS is a novel multi-level approximate logic synthesis framework designed to approximate node simplification [33]. This framework works selectively handles error rate or error magnitude as an optimization constraint.

In [21], DTs and RFs are adopted to represent logic circuits also, providing an attacker viewpoint approach to the generalization. The attacker has a limited care-set from the input and output behavior of the target circuit. The goal would be to provide an approximate circuit from the original as a new product to be sold. Furthermore, their method also includes matching already known logic circuits with the care-set, which provides favorable results when applied on arithmetic circuits. The work [21] considers a limited delay time and different training sizes to create their Boolean networks.

In [34] the DT training is based on a minimum accept-

able accuracy value. If the training does not meet the threshold, another training is applied with data augmentation. The DT nodes are mapped to a MUX structure. Their method is focused on providing small circuits while maintaining acceptable accuracy, most similar to our goal. This approach provided the smallest circuits during the IWLS 2020 contest, and in this work, we compare our method with their solution.

Neural Networks (NNs) may be used on the problem at hand as tools to perform feature selection on the logic care-set, or to perform the mapping from an NN or an MLP to a Boolean network [18]. Two approaches presented in [6] perform the mapping from an NN to the Boolean network. The LUT structure is used as an intermediate during the process. Their experiments demonstrate that this strategy performs well considering the circuit's accuracy, although generating vast Boolean networks. One of the approaches, restrict the adoption of the NN only for logic functions with less than 20 primary inputs [6]. NNs utilization for logic generalization has limitations, mainly due to the difficulty of mapping the NNs structures, such as non-linear activation functions and its size demand due to extravagant arithmetic operations.

It is possible to implement a logic optimization based on the probability of signals in the logic circuit when a care-set is available, accompanied by the logic circuit. This is the case for the problem of generalization for logic functions. The probability of internal nodes in the Boolean network is calculated based on the care-set, and nodes with higher than a certain probability are turned into constants, consequently reducing the size and depth of the Boolean network. This technique is applied in [35] with the AIG representation of deep neural networks that recognize the MNIST dataset. Also, in [18] the technique is explored combined with others approaches to optimize the IWLS 2020 benchmarks [36].

III. CARTESIAN GENETIC PROGRAMMING

CGP is an evolutionary approach proposed as a generalization of Genetic Programming used in the digital circuit's domain [28]. The term *cartesian* in CGP is used because the candidate solutions are composed of a two-dimensional network of nodes. CGP is a stochastic search algorithm that uses an evolutionary approach for exploring the solution's parameter space, usually starting from a random population of candidate solutions. It is a population-based approach that often uses the evolution strategies (1+ λ)-Evolution Strategies (ES) [28] algorithm for searching the parameter space.

CGP is utilized in numerous applications, such as optimizing the learning process of convolutional neural networks [37]. It was also explored as a mechanism in testing, identifying, and repairing software-level faults [38]. Recently, CGP has been employed on the optimization of integrated circuits problems, where each individual is a circuit, represented by a two-dimensional integer matrix describing the functions and the connections among nodes [25, 26, 27, 12].

Figure 1 illustrates the CGP search process, presenting a hypothetical CGP with the XAIG format [14]. This structure comprises all features from the traditional AIG structure with the addition that nodes may be composed of the XOR logic, not only ANDs. Fitness values are presented as accuracy values. In the example, the CGP population contains three individuals, and it is adjusted for three generations searching

for a solution. In our example, generation 0 is created randomly, meaning that nodes have random logic functions, and their inverted or direct connections are randomly set.

Figure 1 also presents the fitness accuracy of each AIG considering the random initialization. The individual with the highest fitness accuracy value (highlighted in yellow) is chosen as the *parent* for the whole next generation. All three individuals in generation 1 are the same ancestor's offspring, which has 52% accuracy, although two individuals have different accuracy values than their ancestor. This is because they underwent mutations (marked in red) during their procreation process. The middle individual in generation 1 had a connection mutated to an inversion, and the right individual had the function of its last node mutated from AND to XOR. These two mutations reflected in lower and higher accuracy, respectively, for each individual. Notice that the right individual in generation 1 is chosen as the parent for the next generation 2. Mutations in generation 2 were not able to achieve any improvement in fitness accuracy. Hence, if the execution went all the way to generation 3, its parent would be the same as generation 2. The example shown in Figure 1 follows a learning configuration of $(1 + 2) - ES$ rule, indicating we have one ancestor and three individuals for each generation. The '+' signal indicates elitism in the generation's procreation, meaning the fittest individual will always be copied to the next generation [39].

Figure 2 presents an example of a CGP implementation with one individual, its set of primitive logic functions, and the individual's genetic code. The CGP structure is represented as a logic circuit, and the CGP Cartesian values are also shown. There is only one line present in the two-dimensional graph. In [40], it is demonstrated that such configuration provides faster convergence when executing the CGP search. Recent work also commonly makes use of such configuration [30]. The CGP implemented in our work uses such configuration of only one line and multiple columns. It is noteworthy that any logic circuit can be represented either in a single line or in a two-dimensional graph. The individuals representation are denoted as *genotype* or genetic coding. This code is given by an array of integers, where a 5-tuple describes each node: the first integer corresponds to the node's logic function, and the other subsequent two pair of integers referencing to its drive-in inputs polarities (direct or inverted input connection) and the input identifier. For example, node 3 is an AND so its first genetic code integer is 0, its first input is direct and comes from the PI 1, so its following integers are 0 and 1, while its second input is negated and comes from PI 2, so its last integers are 1 and 2. The number of primary inputs, primary outputs, and internal CGP nodes is fixed. Notice that the circuit is composed of a functional (*phenotype*) and a non-functional portion, meaning a specific section of the circuit does not influence its primary output.

As demonstrated by [40], the CGP search has a better convergence if phenotypically larger solutions are considered preferred candidates when analyzing the individual's fitness scores. In other words, if, during the evolution, there are two individuals with equal accuracy but different functional sizes, the larger one will be selected as the parent for the next generation. The proposed flows use such a technique by se-

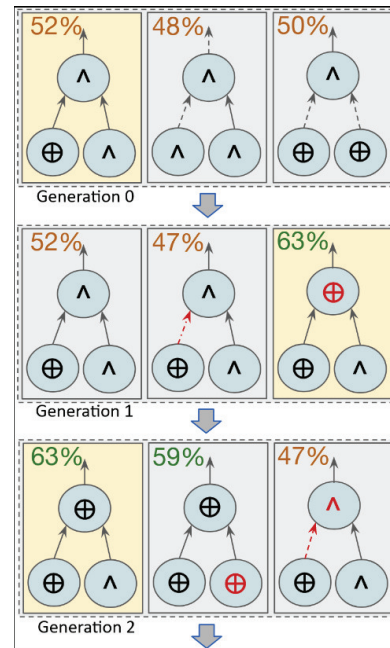


Fig. 1 Evolution development of a hypothetical CGP search.

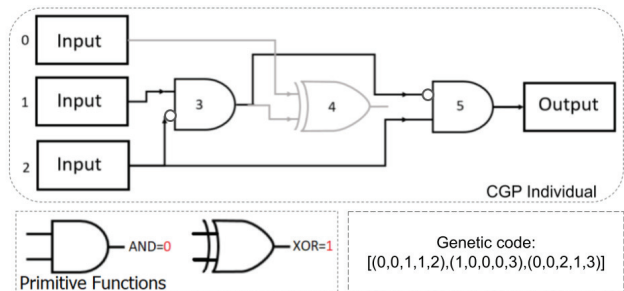


Fig. 2 Single Line CGP Example. Source: adapted from [40].

lecting AIGs with larger functional sizes when a tie in fitness values happens.

IV. PROPOSED CGP-BASED OPTIMIZATION FLOW

We propose an optimization flow based on CGP for logic minimization of Boolean functions that is able to improve accuracy and reduce the number of AIG nodes (size) of circuits. This flow is designed to optimize complete and incomplete truth tables, as well as previously optimized circuits represented in the AIG format. The proposed flow can provide logic minimization solutions from truth tables or fine-tuning alternatives for circuit blocks, considering inaccurate scenarios as on the approximate computing domain, unknown functions employed in intellectual property blocks, or even larger input functions, where the complete truth table can be prohibitive for traditional minimization methods.

Our CGP-based flow is presented in Figure 3. It may operate with a random initialization (pure CGP flow), i.e., a CGP-based only optimization of a complete or incomplete truth table, or with a bootstrapped initialization, where the CGP-based flow will start with a previously optimized circuit in the AIG format (Fine-tuning flow). This initialization option will lead to search configurations that are different

from the bootstrapped ones. We refer to *bootstrap* as the initialization process with a previously synthesized AIG (fine-tuning). The term bootstrap herein employed is related to the bootstrap problem in evolutionary computing [41]. It is noteworthy that our flow can use any circuit described in the AIG format as initialization, making it possible to explore optimized AIGs from different techniques and apply the flow as an additional fine-tuning optimization step.

Considering that larger functional circuits are selected for the sake of better convergence, as proved by [40], smaller circuits are preferred for the sake of area optimization. We designed a two-step flow that considers both requirements. In this mode, the selection process prioritizes faster convergence by selecting larger circuits with the same accuracy. Then, after reaching a reasonable or seemingly local maximum accuracy level, it switches to selecting individuals with smaller sizes, improving circuit area. This way, we explore the two main search options displayed in Figure 3, aiming at obtaining a good balance between accuracy and size.

Our flow restricts the node functions to XORs, ANDs, and Inverters, so we may use AIG or XAIG structures to learn the circuits. After the initialization step is done, the CGP search process takes place, aiming at improving *accuracy* or reducing *size*. The next step is the generation of the optimized circuit and the evaluation of the final optimized circuit about accuracy and size.

To better explain the optimization process, we first introduce three definitions to help the problem formulation. Then, we describe the hyperparameters defined and the possible ranges of these parameters according to the initialization adopted in the CGP-based flow. Finally, we introduce two pseudo-algorithms presented in Algorithms 1 and 2 to detail the general flow proposed and the CGP search.

A. Definitions

Definition 1 (Evolutionary Approach): The evolutionary approach used is a $(1 + 4) - ES$ [39], where one individual generates four new mutated copies of itself. The one with the best performance is selected to create the next generation among this population of five individuals.

Definition 2 (Fitness): The best fitness is given according to the optimization target. In this case, the fitness can be the best accuracy or smallest size depending on the step in the optimization flow, as better described in the pseudo-algorithms 1 and 2.

Definition 3 (Mutations): Variation is added to the individuals through mutations, which applies random reconections on the AIG nodes based on a mutation rate variable. The mutation rate is under optimization according to the $1/5^{th}$ rule [42], in which the mutation rate value varies jointly with the proportion of individuals being better or worse than their ancestor.

B. Main Hyperparameters

We execute the CGP-based flow with three main hyperparameters: (i) **The number of generations**: multiple values up to 100,000 (for size and accuracy). However, it could be customized by the designer according to the circuit to be optimized; (ii) **Two logic structures available**: AIG and XAIG; (iii) **Primary output check**: the option to check for

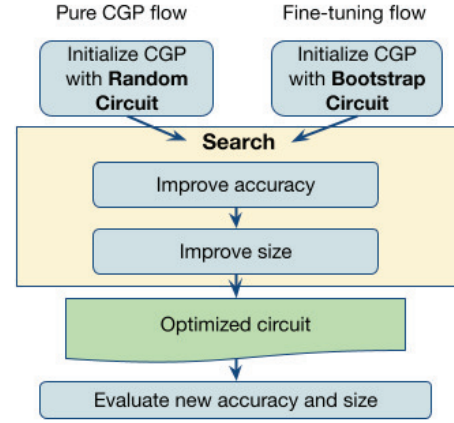


Fig. 3 Proposed CGP-based flow.

all nodes as a possible primary output during every training generation. This is done because an intermediary AIG node could have higher accuracy compared to the actual primary output. This item is a computationally intensive strategy, but it leads to better fitness results in some cases.

The CGP-based flow runs exhaustively, following the constraints of the three main hyperparameters. In addition to these, there are four other hyperparameters dependent on the initialization type. These are presented in Table I. There is one size configuration in the fine-tuning flow where the learning AIG is twice the original one. For each node in the original AIG, a non-functional node is incremented to the learning model, increasing the search space. Due to the bootstrap strategy, the training set must be split among the Pure CGP and the fine-tuning bootstrapping method to provide the initial population. This way, the training set is split in a 40%-40%/20% format, meaning half the training set is used by the initialization method and the other half by the CGP while leaving 20% of the dataset for validation with both methods.

For a random initialization, multiple hyperparameters alternatives are explored. The training set does not have to be split, so it is fully employed in the evolutionary process. Instead of evaluating each individual with the whole training set, an estimate is built using mini-batches. The examples from the training set are randomly selected to compose the batch. Mini-batches are kept unchanged by a given number of generations determined by the *change each* parameter, meaning that the examples used for evaluating individuals change with a frequency given by this parameter. A previous study showed that this technique could lead to the synthesis of more robust solutions, i.e., that generalize well to unseen instances [43]. The random initialization could also use the complete training set as if the batch size was the same as the training set available.

Table I. Hyperparameters dependent on initialization

Parameter	Fine-Tuning	Pure CGP
# ANDs	2x Original # ANDs	500 or 5000
Train/Test(%)	40-40/20	80/20
Batch size	Disabled	1024 or disabled
Change each	Not Applicable	1000 or 2000

C. General Flow and CGP Search Pseudo-Algorithms

The general optimization flow is described in Algorithm 1. The algorithm inputs comprises an AIG file, a truth table file in PLA format, and four parameters defined as integer values to represent the hyperparameter configuration: (i) AIG_{size} : defines the graph size that represents the single line CGP; (ii) B_{size} : defines the size of mini-batch if the mini-batches approach is active; (iii) CE_{size} : defines the frequency of generations that the mini-batches will be changed when mini-batches approach is active; (iv) $Gens_{size}$: limits the number of generations that the CGP will run.

The circuit and the truth table information are processed by reading the AIG and PLA input files (lines 1 and 2). In lines 3 and 4, the algorithm verifies whether the provided AIG is empty. In case it is, the flow will create a new AIG with random connections. Its number of nodes is defined by the input parameter AND_{size} and its number of primary inputs (PIs) indicated by the truth table. Also, the flow performs the CGP search with increased stochasticity with training mini-batches. Otherwise, if a previously synthesized AIG is provided to the flow (lines 5 to 7), it will execute the CGP search with much less stochasticity. The search disables the mini-batch training option (line 6), i.e., using the complete training set and only fine-tuning the previous AIG. Line 8 executes the CGP search by calling the function defined in Algorithm 2, with the accuracy optimization flag as indicated by the parameter $OptAcc$. Line 10 also executes the CGP search, but this time it attempts to improve the AIG's size, as indicated by the parameter $OptSize$. When optimizing either the accuracy of a previous existing AIG or size in any situation, the mini-batch option is disabled, as indicated by lines 6 and 9. The flow's output is the optimized function in AIG format.

Algorithm 2 presents the CGP search in detail. Some initialization steps (lines 2 to 9) are made before the search itself (lines 10 to 21). The batch size B_{size} is checked during preparation steps, and at the end of each iteration (lines 2 and 18), when B_{size} is greater than zero, the CGP search evaluates the circuits on subsets of the training truth table, which we call mini-batches. The size of mini-batches is given by the parameter B_{size} itself and is redefined at CE_{size} (lines 19 and 20). When the circuits are evaluated with mini-batches extracted from the training set, the evaluation process will not use the rows outside of that particular mini-batch for as many generations as the *change each* value. This adds stochasticity to the search, further exploring the search space and avoiding local optima.

When the batch size B_{size} is zero, the size of the batch is the same as the size of the provided truth table, meaning the complete training set will be used (lines 4 and 5). In this case, the training batch will not change for CE_{size} generations. In other words, the *change each* value is disabled in this situation. When the CGP search runs to optimize the size of the AIG, it also does not make use of mini-batches since its utilization would provide a search too divergent from the original AIG, degrading its accuracy. Because of that, line 9 in Alg. 1 disables the mini-batches.

The search population is stored in the list *Popul*, initialized in line 8 with the AIG provided as input parameter. This indi-

Algorithm 1: Proposed CGP-based flow

```

Input:  $AIG_{file}$            // Initial AIG file
Input:  $TT_{file}$           // Truth table file
Input:  $AIG_{size}$           // Graph size
Input:  $B_{size}$             // Mini-batch size
Input:  $CE_{size}$           // Change each size
Input:  $Gens_{size}$         // Number of generations
Output:  $AIG_{opt}$          // Optimized circuit

1  $AIG \leftarrow \text{Read } AIG_{file}$ 
2  $TT \leftarrow \text{Read } TT_{file}$ 
3 if  $AIG$  is empty then
4    $AIG \leftarrow \text{New graph with random connections, with}$ 
    $AIG_{size}$  nodes and PIs the same as  $TT$ .
5 else
6    $B_{size} \leftarrow 0$  // Disable MiniBatches for
   accuracy fine-tuning
7 end
8  $AIG_{opt} \leftarrow \text{cgpSearch}(AIG, TT, B_{size}, CE_{size},$ 
    $Gens_{size}, OptAcc)$ 
9  $B_{size} \leftarrow 0$  // Disable MiniBatches for size
   optimization
10  $AIG_{opt} \leftarrow \text{cgpSearch}(AIG_{opt}, TT, B_{size}, CE_{size},$ 
    $Gens_{size}, OptSize)$ 
11 return  $AIG_{opt}$ 

```

Algorithm 2: CGP Search

```

1 Function  $\text{cgpSearch}(AIG, TT, B_{size}, CE_{size},$ 
    $Gens_{size}, OptChoice)$ 
2   if  $B_{size} > 0$  then
3      $MiniBatch \leftarrow \text{Random set of } B_{size} \text{ rows from}$ 
      $TT$ 
4   else
5      $MiniBatch \leftarrow \text{Complete } TT$ 
6   end
7    $Popul \leftarrow \text{new list}$  // Population
8    $Popul.append(AIG)$ 
9   Evaluate individual  $Popul[0]$  based on  $MiniBatch$ 
10  for  $count \leftarrow 0$  to  $Gens_{size}$  do
11    Copy parent  $Popul[0]$  to  $Popul[1..4]$ 
12    Randomly mutate offspring in  $Popul[1..4]$ 
13    Evaluate offspring in  $Popul[1..4]$  based on
     $MiniBatch$ 
14    if  $OptAcc$  then
15       $Popul[0] \leftarrow \text{highest accuracy and larger}$ 
      individual in  $Popul$ .
16    else if  $OptSize$  then
17       $Popul[0] \leftarrow \text{highest accuracy and smaller}$ 
      individual in  $Popul$ .
18    if  $B_{size} > 0$  then
19      if  $count \bmod CE_{size}$  is 0 then
20         $MiniBatch \leftarrow \text{Random set of } B_{size}$ 
        rows from  $TT$ 
21  end
22  return  $AIG_{opt}$ 

```

vidual is evaluated in line 9 and will be the parent of the first generation. The search will run for as long as the number of generations indicated by the parameter $Gens_{size}$ (line 10). Each generation starts by replicating the parent four times as new offspring. As shown in line 11, every individual in positions 1 to 4 in the population list is forgotten. Lines 12 and 13 randomly modify the recently replicated offspring and then

evaluate them to discover their new accuracy and functional size. The latter process is realized by depth-first search on the AIG. Any dangling AND node will not be visited, meaning the depth first search can reveal the mutated offspring's functional size and logic depth while calculating the new accuracy value.

After mutations and re-evaluations are performed, we select the individual with the best performance to be the parent of the future generation, placing it on the first position of the population list (*Popul[0]*). Lines 14 to 17 show the CGP search behavior when optimizing accuracy and when optimizing the size. Notice it is a subtle difference between them. When optimizing accuracy, we prefer AIGs with larger functional sizes since larger individuals tend to converge better a solution by extending the search space [40]. This preference is performed by breaking ties when ordering the individuals based first on their accuracy and secondly on their size. When we want to optimize size, we prefer AIGs with smaller functional sizes. This way, we sacrifice accuracy. Notice that in either case, the search will never select an individual with lesser accuracy than its current parent, meaning that even if focusing on optimizing size, there is no accuracy sacrifice. Actually, there is still the possibility of improving accuracy during a size-focused search. Furthermore, we realized experiments by introducing a simulated annealing approach, although they did not provide good convergence.

The Bootstrap capability brought by the CGP requires sharing the training set available with other methods used for initializing the AIG. In this work, we tested the initialization with DTs implemented using Sci-kit Learn [20], as well as with Sum of Products (SOP) from the Espresso optimization [3]. The Espresso and DT initialization usually generate AIG models with close to 100% accuracy for the training set. Sometimes, the results present overfitting, mainly for AIGs created by the Espresso technique.

D. Implementation Details

The CGP was implemented in C++ with object-oriented paradigm, the code was compiled using GCC 7.5. Hundreds of thousands of AIGs were generated by varying the previously presented hyperparameters and replicating the experiments with random seeds to find better solutions. In contrast, the flow always maintains the best option.

The fitness evaluation process is vital for CGP learning. However, this is the bottleneck function on execution time. The fitness evaluation of each individual is parallelized with threads, since truth table lines are independent. Thus, the propagation of each input combination from the truth table is executed in parallel. Also, we further adopt parallelism by exploiting the fact that a *long int* is composed of 64 bits, propagating 64 combinations simultaneously along with the AIG, on top of the thread implementation.

V. EVALUATION

To evaluate our CGP-based flow and optimization methods, we observe the accuracy, the number of nodes, the logic depth, and we consider a new metric as a figure of merit (FoM) that demonstrates the relationship between accuracy and size. This FoM is defined in Equation 1, where *ACC* is

the accuracy of an AIG, and *SIZE* is its number of nodes. Using the proposed FoM, larger circuits are penalized even if they lead to the same accuracy. This is a simple way of following Occam's Razor principle of prioritizing more straightforward solutions whenever possible [44].

$$FoM = \frac{ACC}{SIZE} \quad (1)$$

Experiments were executed on the *Emulab Utah* [45] research cluster, utilizing multiple Dell Poweredge R430 nodes equipped with two 2.4 GHz, 64-bit, 20MB cache, 8-Core Xeon processors, 64 GB DDR4 RAM, operating under an Ubuntu 18.04. We ran our flow with up to 100,000 generations on smaller instances and 10 thousand on larger ones, nearly the same amount of generations was performed for size or accuracy. We executed our experiments with the benchmark suite provided by the IWLS 2020 contest [36]. Table II presents the benchmark, which is composed of 100 incomplete functions, each with one primary output. Each function comprises three datasets: train, validation, and test, provided as truth tables in PLA format. Each dataset is composed of 6400 inputs from the complete truth table. We utilize the *train and validation datasets* to train the CGP and validate the solution, and the *test set* to evaluate the comparing experiments and all of the accuracy values presented herein are based on this latter set.

Table II. Benchmark information. Adapted from: [6] and [21].

Func.	Logic type	# Inputs		Logic Domain
		Min	Max	
00-09	Adders	32	512	Arithmetic
10-19	Dividers, Remainders	32	512	Arithmetic
20-29	Multipliers	16	256	Arithmetic
30-39	Comparators	20	200	Arithmetic
40-49	Square-root	10	256	Arithmetic
50-59	PicoJava design	16	394	Random
60-74	MCNC benchmarks	16	52	Random
75-79	Symmetric functions	16	16	Random
80-89	MNIST	196	196	ML
90-99	CIFAR-10	768	768	ML

The circuits are a collection of three different logic domains: arithmetic, random logic, and image recognition. The first 50% of the benchmark suite is composed of single bits extracted from arithmetic logic functions. The benchmark also includes ML problems of image classification based on MNIST and CIFAR-10, totaling 20% of the benchmark suite. 30% of it are function outputs extracted from previous benchmark suites such as MCNC [46]. The largest functions are the benchmarks 80 to 99, that are derived from the MNIST and CIFAR, where the 6400 inputs represent only a tiny portion of all possible combinations of 2^{196} on MNIST and 2^{768} on CIFAR benchmarks, raising the difficulty to infer these functions. For a further inquiry on the benchmark suite, reference [21] provides an overview on the correlation of the functions' inputs.

A. Improvement from initial solution

Table III shows the improvements obtained with the CGP-based flow regarding the number of ANDs and accuracy considering the fine-tuning flow. In this experiment, 9% of the

cases are initialized with a previously function optimized by Espresso, 81% with DT logic minimization solution [20], and 10% with a randomly initialized CGP. The bootstrap or random initialization decision considers the AIG accuracy of the original Espresso and DT AIGs, selecting the model with the highest accuracy. If the accuracy of the previous optimized circuits is below a threshold defined as 50%, the flow ignores the AIG input and defines a new random initialization to try to overcome the inference challenge of the benchmark.

The results show that CGP-based flow optimizes the number of ANDs for bootstrapped populations without compromising their accuracy. We observed that CGP-based flow can reduce the number of ANDs in 33.9% for bootstrapped initialization with Espresso, and 29.06% the bootstrapped from DT. However, we observe an increase on the logic depth in 41% and 92%, respectively. Actually, the logic depth is not an optimization target in the proposed flow, and the logic depth can be correlated to the final delay of the circuit. These results reinforce us to think about the inclusion of the logic depth also in the optimization metrics for the CGP search. The option to random initialize the flow when the input accuracy is inferior than 50% shows an increase on accuracy of 13.82%, reducing the 87.74% the size and improving the logic depth in 65.85%.

Table III. Improvements obtained with the Fine-Tuning flow

Initialization	#ANDs	Acc	Logic depth
Espresso	33.90%	2.16%	-92.36%
Decision Tree	29.06%	0.52%	-41.03%
Random	87.74%	13.82%	65.85%

B. Comparison with Related Work

We selected two solutions from the participants of the IWLS 2020 contest to compare and evaluate the proposed CGP-based flow. These solutions are chosen due to best ac-

curacy [18], and, due to smallest graph size [34]. We compare our results with the circuits (the final AIG files) provided by these two teams for the same benchmarks sets.

The results of accuracy, number of ANDs and logic depth for all the 100 benchmarks evaluated are presented in Figs. 4, 5 and 6, respectively. Our results are presented with blue columns and the related work outcomes are presented in red [18] and yellow [34] lines.

The accuracy results represents a way to evaluate the generalization capability of the methods considering the incomplete information of the truth tables. We can see an evenly distributed accuracy among the comparing methods for almost all the domain of random logic, a good portion of arithmetic (10-19, 30-59), and the CIFAR-10 functions. We can see some functions that [18] was able to perform better. Their method was able to completely solve some arithmetic benchmarks which ours and [34] was not able to. One possible cause of this disparity is mainly due to their approach adopting logic matching prior to executing ML techniques [18]. This technique was also used by another work [21]. Some arithmetic logic seem hard to learn using ML. Also, the solution of [18] provided best results then others on the MNIST functions, showing the advantages of the multi solution mechanism proposed to optimize accuracy.

Observing the number of nodes, we can clearly see how the results of our proposed flow is in between the other two methods [18] and [34] for most of the functions. The [34] solution shows, in general, to be the best approach to generate smaller circuits. Three benchmarks (26, 48, and 78) deserve more attention. These functions seem to be the hardest to learn among all the benchmarks considered, with low reported accuracy independently of the optimization approach adopted. The solution found by [18] was to simplify these functions to constants, i.e., containing zero AND nodes. The definition of the output is done by finding the most frequent constant between 0 or 1, based on the training set. Their validation set indicated the constant method was actually the

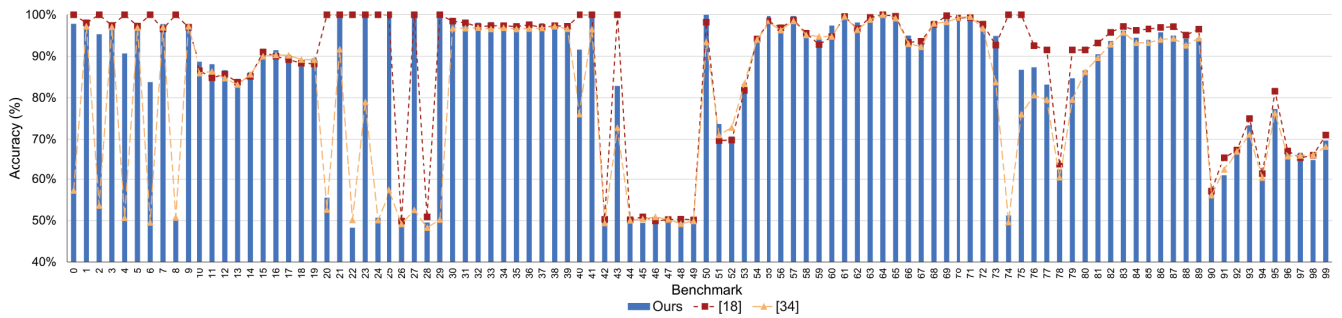


Fig. 4 Accuracy results for all evaluated benchmarks.

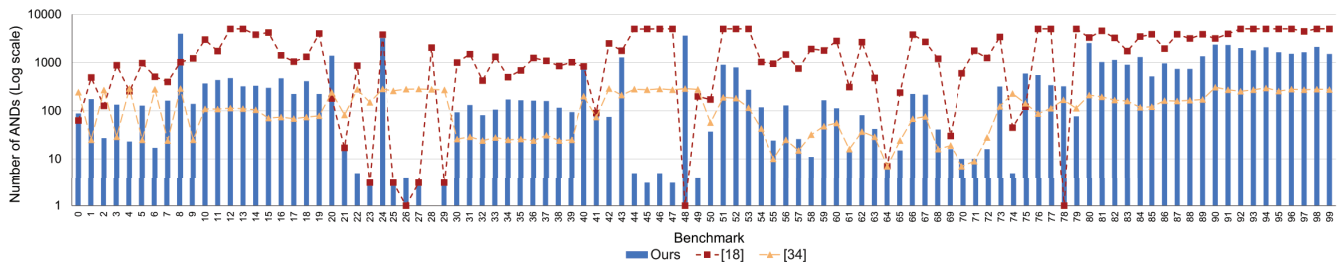


Fig. 5 Number of ANDs for all evaluated benchmarks.

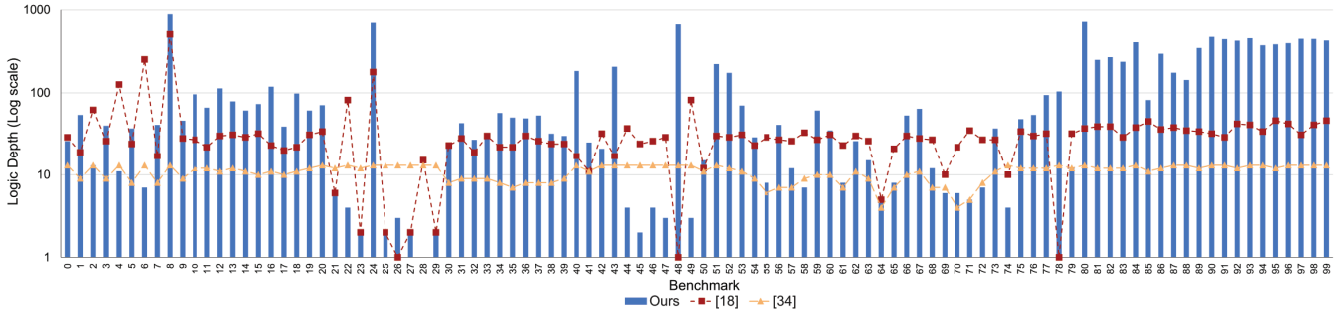


Fig. 6 Logic depth of the evaluated benchmarks.

best for the three mentioned functions. Otherwise, the DT-based approaches define an output function, reporting more AND nodes with similar accuracy.

Other noteworthy cases are the ones that lead to large circuits with high logic depth values. Our solutions for functions 8, 24, 48 present elevated number of ANDs. This behaviour is due to the fact that, during the creation of bootstrapped DTs, multiple configurations were attempted, and the ones that presented small accuracy improvement were the ones with largest tree depth, resulting in larger and deeper AIGs for bootstrapping our CGP-based flow. Figure 6 shows that our favorable solutions relating to accuracy and size, came with the cost of AIGs with longer depths in most of the functions. However, this evaluation makes room to the potential of improve the CGP-based flow to also consider logic depth in the CGP search optimization.

Figure 7 summarizes the accuracy for the provided AIGs showing a boxplot of all the accuracy results for the 100 benchmarks evaluated. We can see how [18] presents higher accuracy values, although their AIGs are usually large. The accuracy of [34] presents a large deviation, with the majority of the functions presenting low accuracy. On average, our solution is 3.78% inferior in accuracy than [18], and presented a larger standard deviation. However the CGP-based solution is 5.74% superior in accuracy to [34] on average.

Now, observing the boxplot presented in Figure 8 summarizing the number of AIG nodes for all functions, we see that, oppositely to the accuracy, [18] provides the largest AIGs with an average size of 2173 nodes, followed by our solution with 561 nodes on average. The AIGs provided by [34] are the smallest, with an average size of 140 nodes. Such characteristic is desirable since it will lead to a cheaper circuit for fabrication, although their AIGs present high error rates that may not be acceptable for some error-tolerant applications. However, compared to the CGP combined optimization flow for accuracy and size, our solution provides AIGs only four times larger in the number of nodes than [34] solution while maintaining an accuracy level closer to [18] (3.78%). In contrast, [34] solution was 9.89% distant in accuracy from [18].

Considering all functions, the best way to solve the challenge appears to be proposing the most number of diversified algorithms as possible. The solution of [34] seems to compose a single approach, and is still able to provide good results when considering both accuracy and size. However, in the next results to be presented, we show that our method excels when taking into account both accuracy and size optimization simultaneously.

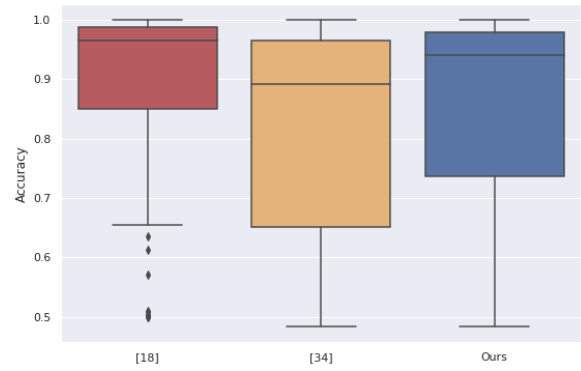


Fig. 7 Distribution of accuracy results achieved by each solution.

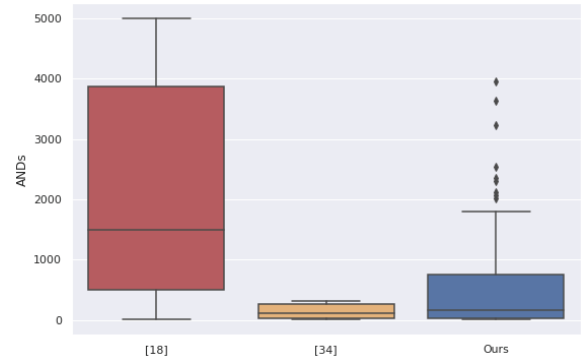


Fig. 8 Distribution of size results achieved by each solution.

The FoM presented in Eq. 1 is used as a metric to analyze the AIG solution regarding accuracy and size trade-off. Figure 9 demonstrates the FoM distribution among the methods. The results for [18] shows the worst FoM in general, with the mean influenced by some few cases with the best FoM outcomes. The results for [34] presents a stable median value compared with our solution, with less variability on their FoM results. Although our solution improves on average the trade-off by 114% compared to [34] solution. Aside from the few outliers from [18], our solution stays ahead on average by 22.6%. We can adjust and focus on both objectives with our flow: good accuracy and reduced size. As our solution does reasonably well on both of them, we can provide decent accuracy values. At the same time, most of our AIGs are small compared to related work, thus, combining with high accuracy we have a slight advantage over the median behavior of [34], with some outliers with good outcomes for the FoM, that elevates the average behavior when directly compared with the [34] results.

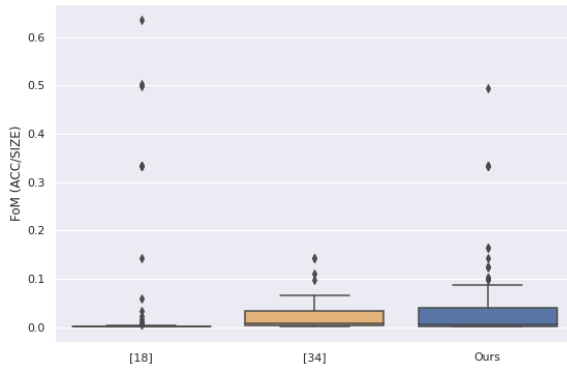


Fig. 9 Distribution of the figure of merit - accuracy per size.

Runtime is an extra metric to be considered in a Boolean logic minimization algorithm. The CGP-based flow is computationally expensive due to time for convergence and mainly to fitness evaluation, where the latter is the algorithm bottleneck. The runtime and accuracy are correlated to the number of generations executed. In our experiment environment, the runtime to process one generation of an individual with 768 primary inputs and 4,114 nodes is about 0.85 seconds. It takes one day to run 100,000 generations on this worst case. Direct runtime comparison with [34] and [18] is prohibitive since we have only their final AIG solutions. The comparison with Espresso and other traditional methods could be biased because most of these methods cannot deal with logic functions larger than 16 inputs and are not developed to consider partial truth tables. For a fair comparison with these traditional logic optimization methods, in the future steps, we could execute the CGP flow to optimization of standard benchmarks [46], for example.

VI. CONCLUSION

The CGP-based flow is a promising technique for logic optimization when time and computational resources are available. It may further improve the solutions provided by any other techniques, and it can obtain improvements based on a predetermined objective. In this work, we presented results enhancing the accuracy and size of approximate logic circuits with the proposed CGP-based flow.

We will investigate the inclusion of logic depth as a third CGP search optimization step for future work. The adoption of multi optimization functions on the CGP fitness value did not provide beneficial results because the two objectives, accuracy and size, are conflicting. The functional size of the circuits is directly proportional to their evolvability in terms of accuracy. Thus, adopting multi optimization steps shows to be the better approach for this CGP flow. The proposed optimization flow may imply an extra runtime, mainly compared to fast minimization approaches. However, this cost is a trade-off to obtain higher accuracy and better logic optimization for approximate circuits synthesis. To speed up our implementation, we plan to explore strategies to avoid redundant mutations as proposed by [47], the idea is that a considerable amount of mutations does not influence the functional part of the individual, so a reevaluation is completely wasteful.

ACKNOWLEDGEMENTS

This study was financed in part by the *Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES)* - Finance Code 001 and by the *Conselho Nacional de Desenvolvimento Científico e Tecnológico*. The authors also would like to thank the authors from [34] and [18] to send us their optimized circuits.

REFERENCES

- [1] M. Karnaugh, "The map method for synthesis of combinational logic circuits," *Transactions of the American Institute of Electrical Engineers, Part I: Communication and Electronics*, vol. 72, no. 5, pp. 593–599, 1953. [Online]. Available: <https://doi.org/10.1109/TCE.1953.6371932>
- [2] W. V. Quine, "A way to simplify truth functions," *The American Mathematical Monthly*, vol. 62, no. 9, pp. 627–631, 1955. [Online]. Available: <https://doi.org/10.1080/00029890.1955.11988710>
- [3] R. L. Rudell and A. Sangiovanni-Vincentelli, "Multiple-valued minimization for pla optimization," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 6, no. 5, pp. 727–750, 1987. [Online]. Available: <https://doi.org/10.1109/TCAD.1987.1270318>
- [4] R. K. Brayton, G. D. Hachtel, C. T. McMullen, and A. L. Sangiovanni-Vincentelli, "Logic Minimization Algorithms for VLSI Synthesis," *The Kluwer International Series in Engineering and Computer Science*, vol. 2, p. 1–194, 1984. [Online]. Available: <https://doi.org/10.1007/978-1-4613-2821-6>
- [5] O. Coudert and T. Sasao, *Two-Level Logic Minimization*. Boston, MA: Springer US, 2002, pp. 1–27. [Online]. Available: https://doi.org/10.1007/978-1-4615-0817-5_1
- [6] S. Rai and et al., "Logic synthesis meets machine learning: Trading exactness for generalization," in *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2021. [Online]. Available: <https://doi.org/10.23919/DATE51398.2021.9473972>
- [7] P. A. Beerel and M. Pedram, "Opportunities for machine learning in electronic design automation," in *2018 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2018, pp. 1–5. [Online]. Available: <https://doi.org/10.1109/ISCAS.2018.8351731>
- [8] A. Fayyazi et al., "Deep learning-based circuit recognition using sparse mapping and level-dependent decaying sum circuit representations," in *2019 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2019, pp. 638–641. [Online]. Available: <https://doi.org/10.23919/DATE.2019.8715251>
- [9] W. L. Neto et al., "Lsoracle: a logic synthesis framework driven by artificial intelligence," in *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 2019, pp. 1–6. [Online]. Available: <https://doi.org/10.1109/ICCAD45719.2019.8942145>
- [10] I. Scarabottolo et al., "Approximate Logic Synthesis: A Survey," *Proceedings of the IEEE*, pp. 1–19, 2020. [Online]. Available: <https://doi.org/10.1109/JPROC.2020.3014430>
- [11] H. B. Barua and K. C. Mondal, "Approximate computing: A survey of recent trends—bringing greenness to computing and communication," *Journal of The Institution of Engineers (India): Series B*, pp. 1–8, 2019. [Online]. Available: <https://doi.org/10.1007/s40031-019-00418-8>
- [12] A. Berndt, B. A. de Abreu, I. S. Campos, B. Lima, M. Grellert, J. T. Carvalho, and C. Meinhardt, "Accuracy and size trade-off of a cartesian genetic programming flow for logic optimization," in *Proceedings of the 34th Symposium on Integrated Circuits and Systems Design*, ser. SBCCI '21, 2021. [Online]. Available: <https://doi.org/10.1109/SBCCI53441.2021.9529968>

- [13] A. Mishchenko, S. Chatterjee, and R. Brayton, "DAG-aware AIG rewriting: a fresh look at combinational logic synthesis," in *2006 43rd ACM/IEEE Design Automation Conference*, 2006, pp. 532–535. [Online]. Available: <https://doi.org/10.1145/1146909.1147048>
- [14] I. Háleček, P. Fišer, and J. Schmidt, "Towards AND/XOR balanced synthesis: Logic circuits rewriting with XOR," *Microelectronics Reliability*, vol. 81, pp. 274–286, 2018. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0026271417305899>
- [15] S. Shirinzadeh, M. Soeken, P. Gaillardon, and R. Drechsler, "Fast logic synthesis for RRAM-based in-memory computing using Majority-Inverter Graphs," in *2016 Design, Automation Test in Europe Conference Exhibition (DATE)*, March 2016, pp. 948–953. [Online]. Available: <https://ieeexplore.ieee.org/document/7459444?arnumber=7459444>
- [16] H. Riener, W. Haaswijk, A. Mishchenko, G. De Micheli, and M. Soeken, "On-the-fly and DAG-aware: Rewriting Boolean Networks with Exact Synthesis," in *2019 Design, Automation Test in Europe Conference Exhibition (DATE)*, 2019, pp. 1649–1654. [Online]. Available: <https://doi.org/10.23919/DATE.2019.8715185>
- [17] S. Chatterjee, "Learning and memorization," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, J. Dy and A. Krause, Eds., vol. 80. PMLR, 10–15 Jul 2018, pp. 755–763. [Online]. Available: <https://proceedings.mlr.press/v80/chatterjee18a.html>
- [18] Y. Miyasaka, X. Zhang, M. Yu, Q. Yi, and M. Fujita, "Logic Synthesis for Generalization and Learning Addition," in *2021 Design, Automation Test in Europe Conference Exhibition (DATE)*, 2021, pp. 1032–1037. [Online]. Available: <https://doi.org/10.23919/DATE51398.2021.9474169>
- [19] R. D. Peralta, J. P. Nespolo, P. F. Butzen, M. L. Kolberg, and A. I. Reis, "A method to join the On-set and Off-set of an incompletely boolean function into a single BDD," in *2021 34th SBC/SBMicro/IEEE/ACM Symposium on Integrated Circuits and Systems Design (SBCCI)*, 2021, pp. 1–6. [Online]. Available: <https://doi.org/10.1109/SBCCI53441.2021.9529985>
- [20] B. A. de Abreu, A. Berndt, I. S. Campos, C. Meinhardt, J. T. Carvalho, M. Grellert, and S. Bampi, "Fast logic optimization using decision trees," in *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2021, pp. 1–5. [Online]. Available: <https://doi.org/10.1109/ISCAS51556.2021.9401664>
- [21] W. Zeng, A. Davoodi, and R. O. Topaloglu, "Lorax: Machine learning-based oracle reconstruction with minimal i/o patterns," in *2021 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, 2021, pp. 126–131. [Online]. Available: <https://doi.org/10.1109/ISVLSI51109.2021.00033>
- [22] S.-Y. Lee, H. Riener, A. Mishchenko, R. K. Brayton, and G. De Micheli, "A Simulation-Guided Paradigm for Logic Synthesis and Verification," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, pp. 1–1, 2021. [Online]. Available: <https://doi.org/10.1109/TCAD.2021.3108704>
- [23] A. M. R. Brayton, "Scalable logic synthesis using a simple circuit structure," in *Proc. IWLS*, vol. 6, 2006, pp. 15–22. [Online]. Available: <https://www.semanticscholar.org/paper/Scalable-Logic-Synthesis-using-a-Simple-Circuit-Mishchenko-Brayton/28afc93b93a39ba29a96545b1724be40ed9e2da7>
- [24] J. Miller, P. Thomson, T. Fogarty, and I. Ntroduction, "Designing Electronic Circuits Using Evolutionary Algorithms. Arithmetic Circuits: A Case Study," *Genetic Algorithms and Evolution Strategies in Engineering and Computer Science*, 10 1999. [Online]. Available: https://www.researchgate.net/publication/2352606_Designing_Electronic_Circuits_Using_Evolutionary_Algorithms_Arithmetic_Circuits_A_Case_Study
- [25] D. Hodan, V. Mrazek, and Z. Vasicek, "Semantically-oriented mutation operator in cartesian genetic programming for evolutionary circuit design," *Genetic Programming and Evolvable Machines*, vol. 22, no. 4, pp. 539–572, 2021. [Online]. Available: <https://doi.org/10.1007/s10710-021-09416-6>
- [26] J. Kocnova and Z. Vasicek, "EA-based resynthesis: an efficient tool for optimization of digital circuits," *Genetic Programming and Evolvable Machines*, pp. 1–33, 2020. [Online]. Available: <https://doi.org/10.1007/s10710-020-09376-3>
- [27] J. Kocnová and Z. Vasicek, "Resynthesis of logic circuits using machine learning and reconvergent paths," in *2021 24th Euromicro Conference on Digital System Design (DSD)*, 2021, pp. 69–76. [Online]. Available: <https://doi.org/10.1109/DSD53832.2021.00020>
- [28] J. F. Miller, "Cartesian genetic programming: its status and future," *Genetic Programming and Evolvable Machines*, pp. 1–40, 2019. [Online]. Available: <https://doi.org/10.1007/s10710-019-09360-6>
- [29] A. J. Turner and J. F. Miller, "Cartesian genetic programming: Why no bloat?" in *European Conference on Genetic Programming*. Springer, 2014, pp. 222–233. [Online]. Available: https://doi.org/10.1007/978-3-662-44303-3_19
- [30] A. Manazir and K. Raza, "Recent developments in cartesian genetic programming and its variants," *ACM Computing Surveys (CSUR)*, vol. 51, no. 6, pp. 1–29, 2019. [Online]. Available: <https://doi.org/10.1145/3275518>
- [31] A. Wendler and O. Keszocze, "A fast BDD Minimization Framework for Approximate Computing," in *2020 Design, Automation Test in Europe Conference Exhibition (DATE)*, 2020, pp. 1372–1377. [Online]. Available: <https://doi.org/10.23919/DATE48585.2020.9116296>
- [32] A. Stempkovsky, D. Telpukhov, and R. Solovyev, "Synthesis of approximate combinational circuits based on logic regression approach," in *2020 IEEE East-West Design Test Symposium (EWDTS)*, 2020, pp. 1–5. [Online]. Available: <https://doi.org/10.1109/EWDTS50664.2020.9224981>
- [33] Y. Wu and W. Qian, "ALFANS: Multilevel Approximate Logic Synthesis Framework by Approximate Node Simplification," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 7, pp. 1470–1483, 2020. [Online]. Available: <https://doi.org/10.1109/TCAD.2019.2915328>
- [34] V. Tenace and A. Calimera, "Inferential Logic: a Machine Learning Inspired Paradigm for Combinational Circuits," in *2018 IFIP/IEEE International Conference on Very Large Scale Integration (VLSI-SoC)*, 2018, pp. 149–154. [Online]. Available: <https://doi.org/10.1109/VLSI-SoC.2018.8644808>
- [35] A. Berndt, C. Meinhardt, A. I. Reis, and P. F. Butzen, "Exploring Constant Signal Propagation to Optimize Neural Network Circuits," in *Proceedings of the 34th Symposium on Integrated Circuits and Systems Design*, ser. SBCCI '21, 2021. [Online]. Available: <https://doi.org/10.1109/SBCCI53441.2021.9529971>
- [36] IWLS 2020 programming contest, "IWLS 2020 Benchmarks," 2020. [Online]. Available: <https://github.com/iwls2020-lsml-contest/iwls2020-lsml-contest>
- [37] M. Suganuma *et al.*, "Evolution of Deep Convolutional Neural Networks Using Cartesian Genetic Programming," *Evolutionary Computation*, vol. 28, no. 1, pp. 141–163, 03 2020. [Online]. Available: <https://doi.org/10.1162/evco.a.00253>
- [38] A. Khalilian, A. Baraani-Dastjerdi, and B. Zamani, "Cgenprog: Adaptation of cartesian genetic programming with migration and opposite guesses for automatic repair of software regression faults," *Expert Systems with Applications*, vol. 169, p. 114503, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0957417420311477>

- [39] N. Hansen, D. V. Arnold, and A. Auger, “Evolution strategies,” in *Springer handbook of computational intelligence*. Springer, 2015, pp. 871–898. [Online]. Available: https://doi.org/10.1007/978-3-662-43505-2_44
- [40] N. Milano, P. Pagliuca, and S. Nolfi, “Robustness, evolvability and phenotypic complexity: insights from evolving digital circuits,” *Evolutionary Intelligence*, vol. 12, no. 1, pp. 83–95, 2019. [Online]. Available: <https://doi.org/10.1007/s12065-018-00197-z>
- [41] J.-B. Mouret and S. Doncieux, “Overcoming the bootstrap problem in evolutionary robotics using behavioral diversity,” in *2009 IEEE Congress on Evolutionary Computation*. IEEE, 2009, pp. 1161–1168. [Online]. Available: <https://doi.org/10.1109/CEC.2009.4983077>
- [42] B. Doerr and C. Doerr, “Optimal parameter choices through self-adjustment: Applying the 1/5-th rule in discrete settings,” in *Proceedings of the 2015 Annual Conference on Genetic and Evolutionary Computation*, 2015, pp. 1335–1342. [Online]. Available: <https://doi.org/10.1145/2739480.2754684>
- [43] J. Carvalho, N. Milano, and S. Nolfi, “Evolving Robust Solutions for Stochastically Varying Problems,” in *2018 IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2018, pp. 1–8. [Online]. Available: <https://doi.org/10.1109/CEC.2018.8477811>
- [44] P. Domingos, “The role of Occam’s razor in knowledge discovery,” *Data mining and knowledge discovery*, vol. 3, no. 4, pp. 409–425, 1999. [Online]. Available: <https://doi.org/10.1023/A:1009868929893>
- [45] B. White *et al.*, “An Integrated Experimental Environment for Distributed Systems and Networks,” in *OSDI02*. Boston, MA: USENIXASSOC, Dec. 2002, pp. 255–270. [Online]. Available: <https://doi.org/10.1145/844128.844152>
- [46] S. Yang, *Logic synthesis and optimization benchmarks user guide: version 3.0*. Microelectronics Center of North Carolina (MCNC), 1991. [Online]. Available: <https://www.semanticscholar.org/paper/Logic-Synthesis-and-Optimization-Benchmarks-User-Yang/4cd302de6b2e911c980c9d5fcc009e8215c2c560>
- [47] B. W. Goldman and W. F. Punch, “Reducing Wasted Evaluations in Cartesian Genetic Programming,” in *Genetic Programming*, K. Krawiec, A. Moraglio, T. Hu, A. Ş. Etaner-Uyar, and B. Hu, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 61–72. [Online]. Available: https://doi.org/10.1007/978-3-642-37207-0_6